Intelligent Greenhouse

Members:

- Indhira Dannae Ramirez Perez Gao
- Hugo Jeremy Armejo Pascual
- Bryan Martin Jaramillo Olortegui

1. Abstract

Intellect Greenhouse is an innovative, self-regulating system developed for the autonomous management of temperature and humidity. Utilizing a hierarchical control architecture, which integrates constellation, will, reason, understanding, presentation, intellect, and sensory subsystems, the project achieves precise temperature regulation through dynamic heater activation. This system provides a robust solution for maintaining optimal environmental conditions in greenhouse environments, minimizing manual intervention.

2. Description of the System

2.1. Mission

"Maintain optimal conditions for plant growth"

2.2. System

- Will System:

  Responsible for executing actions. The purpose of the system is the mission that in this case is "Maintain optimal conditions for plant growth" (temperature). As this is a small implantation, we focus the mission in the control of the temperature in plants, only in the case the temperature is too low that the heater needs to be turned on until it reaches a limit of temperature and then turned off after it reaches another limit of temperature. Also, turn on or off the sensor if it is necessary in the case a sensor is faulty.

- Reason System:

  Ensures the greenhouse operates within predefined rules, preventing actions that could lead to energy waste or system conflict. These rules could be obtained from different sources like the maximum and minimum temperature a sensor can read or a maximum time a heater can be turned on. In this system if a sensor is marked as outlier, its reading must be ignored because we only use these rules in sensors that are working well. In our implementation, the Reason System only reads the values of the sensors but does not

have anything else, it is because we focus on one use case and selecting proper rules would be a complicated task.

- Intellect System:

  Stores all the information needed by the system. In this case it could use a database of past temperatures from different sensors, the failure events of the sensors like which sensor failed and at what time. This information can be use for future anomalies, like knowing that after a certain time a sensor would be more likely to fail, or the average temperature in each season, so if there is a strange value, it could help to find an error.

- Presentation System:

  It permits communication using a standardized language and grammar, ensuring seamless interaction. This common language simplifies communication across all systems, as all exchanges must adhere to the same format.

- Understanding System:

  Determinants the current state of the system. Here we have the detection of the anomalies in the temperature values. So, if there is an anomaly, the system needs to understand if this anomaly is because there is a change in the temperature, if it is to low the heater may need to be turned on, or if there is a faulty sensor and needs to be turned off, because the temperature is calculated with the average of the 3 sensors, turning off the faulty sensor can help with the bad read.

- Sensory System:

  Receives data from the sensors and sends the data within the constellation. For this case we use 3 temperature sensors, but as we don't have the physical sensors, we simulate the value of the sensors, and send them to all the constellation.

- Decision System:

  Receives the state of the system from the Understanding System and makes a decision based on the state, so if the system “understands” that a sensor is failing, the system “decides” that this sensor needs to be turned off, or if all the sensors are working well and the temperature is below a threshold, the “decision” would be to turn on the heater to maintain the temperature of the plants.

- Thought System:

  Employs internal communication mechanisms that facilitate awareness and interaction between its subsystems. In our implementation, this mechanism initiates and connects all subsystems, enabling coordinated and coherent operation.

2.3. Use case

Our use case is to know what happens if the temperature goes below a certain value of temperature, when this happens it could mean 2 possible cases, the first one is that 1 of the 3 sensors reads a wrong temperature value (in this case we have the value of 0 ºC as a bad value) so the average temperature decrease in one moment and we need to detect and turn off the faulty sensor, and the second case is that the 3 sensors read good temperature values but the temperature of the environment decreased, so we need to turn on the heater.

3. Implementation

3.1. Tools

Python was used for the implementation. The libraries used were:

- Tkinter: used for the GUI
- Pubc: used for Public/subscribe protocol that connect all the subsystems of the "thought".
- Networkx: Used to implement the failure graph.
- Threading: Used to run the GUI and the "thought" system in parallel.

3.2. Architecture

3.2.1. Intro

For this system, we have 3 sensors and 1 heater. The digital twin is simulated with global variables that change their values with the input of the GUI.

3.2.1. Architecture

The sensory system, understanding system, reason system, decision system and will system are implemented as classes. All of these classes are inside the thought system class. The thought system class initializes all the subsystems. And everything is executed as the execute function calls the sensory_system_execute which publishes the data sensors. The code is in [this link](#).

```python
class thought_system:
    def __init__(self):
        self.SENSORY_SYS = sensory_system()

        self.UNDERSTANDING_SYS = understanding_system()
        pub.subscribe(self.UNDERSTANDING_SYS.listener_sensory_sys, 'SENSORY_SYS_TOPIC')

        self.REASON_SYS = reason_system()
        pub.subscribe(self.REASON_SYS.listener_sensory_sys, 'SENSORY_SYS_TOPIC')

        self.DECISION_SYS = decision_system()
        pub.subscribe(self.DECISION_SYS.listener_understanding_sys, 'UNDERSTANDING_SYS_TOPIC')

        self.WILL_SYS = will_system()
        pub.subscribe(self.WILL_SYS.listener_decision_sys, 'DECISION_SYS_TOPIC')

    def execute(self):
        for i in range(1000):
            print("executing thought system")
            self.SENSORY_SYS.execute()
            time.sleep(2)
```

3.3. Code explanation

This point explains the logic of how the subsystems were implemented in our use case.

- **Will system**

  The mission of the system is to maintain ideal conditions of temperature. The will system is the responsible to execute the actions, in this case it can execute:

  - turn on / off the heater
  - turn on / off the sensors

- **Reason system**

  The reason system is implemented very briefly in our code, it only receives data from the sensory system. Ideally, in our implementation, the reason system could:

  - If a sensor is marked as outlier, its reading must be ignored.
  - Heating rules.
  - Temperature rules.
  - Sensors maximum and minimum (maximum and minimum value capable of read)

- **Intellect system**

  Stores all the information needed by the system. We implemented this subsystem by creating global variables that can be accessed for every subsystem. It contains:

  - Global minimum temperature

- Global maximum temperature
- The faulty graph
- Information about the nodes of the failure graph

- **Presentation system**

  The presentation system is not explicitly implemented. Nevertheless, the process of receiving a message from another system and interpreting it will be considered as a brute force presentation system.

- **Understanding**

  The pseudocode of the understanding system is at follows:

  1. The understanding system receives data from the sensory system.

  2. The understanding get range of ideal temperatures from the intellect system

  3. The understanding system analice if the sensor data is within the range of temperatures

  4. The understanding system analice if all the sensors data is similar

     4.1. If the sensors do not have similar data:

     - Use back propagation will start with the node "DELTA_TEMPERATURE_VARY"
     - The back propagation will find its predecessors that are: SENSOR01_OUTLIER, SENSOR02_OUTLIER, SENSOR03_OUTLIER as one of the sensors having a bad reading, outlier value, can be the cause of the variation. The back propagation will find which sensor is the outlier and return a status. For example: "SENSOR03_OUTLIER"
     - After receiving the actual status from the back propagation function, the understanding system will publish the status.

     4.2. If the sensors have similar data:

     - It calculates the average of all the working sensors
     - If the average is less that the minimum temperature permitted:
       - The understanding system publish "TURNON HEATER"
     - If the average is more or equal to the maximum temperature permitted:
       - The understanding system publish "TUROFF HEATER"

- **Sensory system**

  The sensory system publishes the temperature register for the three sensors.

```
class sensory_system:
    def execute(self):
        global GLOBAL_TEMPERATURE_SENSOR01, GLOBAL_TEMPERATURE_SENSOR02, GLOBAL_TEMPERATURE_SENSOR03

        GLOBAL_TEMPERATURE_SENSOR01.set(temperature.get())
        GLOBAL_TEMPERATURE_SENSOR02.set(temperature2.get())
        GLOBAL_TEMPERATURE_SENSOR03.set(temperature3.get())

        print("-> Sensor system send: ")
        print("     sensor 1: ", GLOBAL_TEMPERATURE_SENSOR01.get())
        print("     sensor 2: ", GLOBAL_TEMPERATURE_SENSOR02.get())
        print("     sensor 3: ", GLOBAL_TEMPERATURE_SENSOR03.get())
        pub.sendMessage('SENSORY_SYS_TOPIC',
                        arg1=GLOBAL_TEMPERATURE_SENSOR01.get(),
                        arg2=GLOBAL_TEMPERATURE_SENSOR02.get(),
                        arg3=GLOBAL_TEMPERATURE_SENSOR03.get())
```

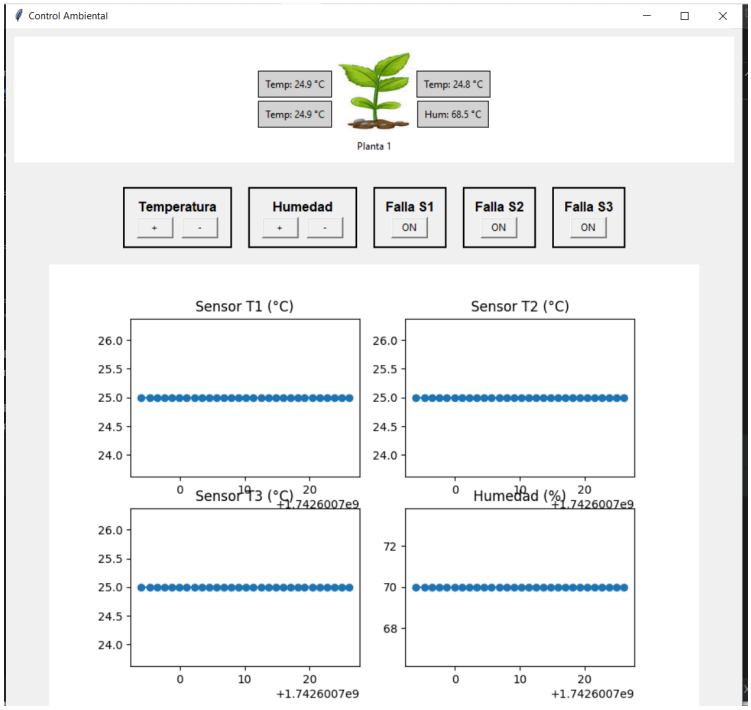
- **Thought system**

  The thought system initializes all the other subsystems: will, reason, intellect, understanding and sensory systems. And connects the subsystems via publisher/subscriber protocol.

4. Demonstration and results

Case 1: Normal conditions

The image depicts a digital twin of our system. The system's functionality is confirmed by the sensory system's continuous data transmission to the understanding system. Since the plant is in ideal condition, the 'will' system remains inactive.

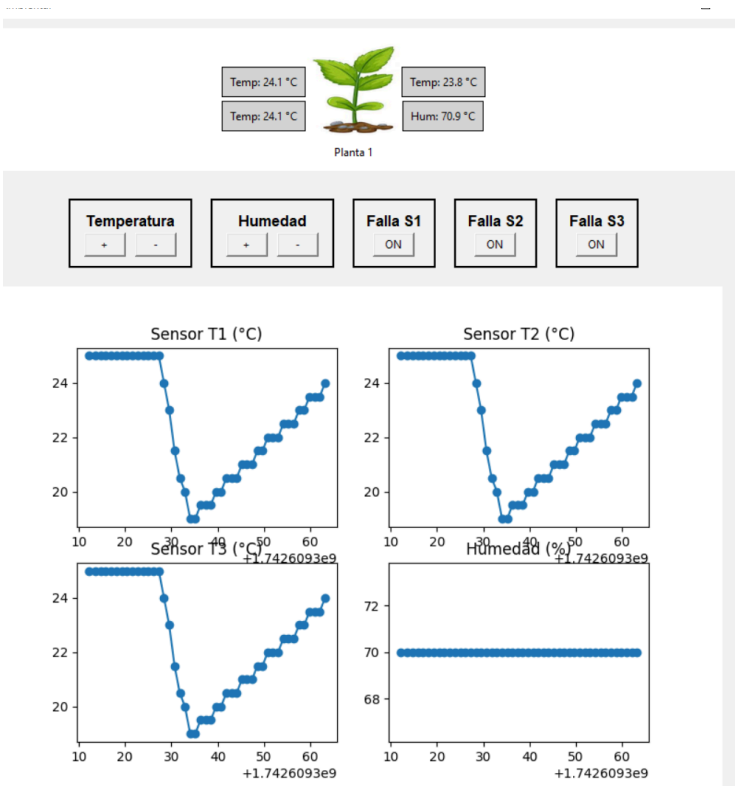



```
executing thought system
-> Sensor system send:
     sensor 1:  25.0
     sensor 2:  25.0
     sensor 3:  25.0
-> Understanding system reads from sensory system

-> Reason system reads from sensory system
```

Case 2: Heater ON

The image displays a digital representation of the greenhouse under ideal conditions. A simulated temperature decrease is detected by the sensors. When the temperature drops below the ideal level, the heater activates, resulting in a temperature increase. This action is triggered because the understanding system recognizes the low temperature, relays this information to the decision system, which then commands the 'will' system to turn on the heater. The 'will' system then executes this command.



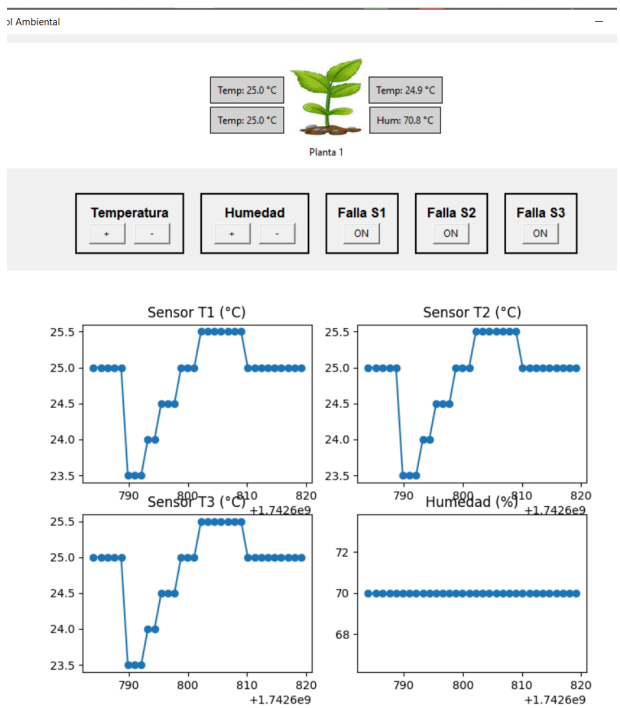
```
executing thought system
-------------------------------------------------------
-> Sensor system send:
     sensor 1:  19.5
     sensor 2:  19.5
     sensor 3:  19.5
-> Understanding system reads from sensory system

-> Desicion system reads from understanding system:   TURNON HEATER
['TURNON', 'HEATER']
-> Decision system send message:  TURNON HEATER
Will system recive  ['TURNON', 'HEATER']
-> Will system turn on heater
-> Reason system reads from sensory system

executing thought system
```

Case 3: Heater OFF

The image demonstrates a temperature control system. When the temperature falls below the setpoint of 23°C, the heating system activates, raising the temperature. Conversely, when the temperature exceeds 23°C, the heater deactivates, allowing the temperature to drop. This prevents potential damage to plants due to overheating. The decision system, recognizing the risk, commands the 'will' system to turn the heater on to off, and the 'will' system executes the command.

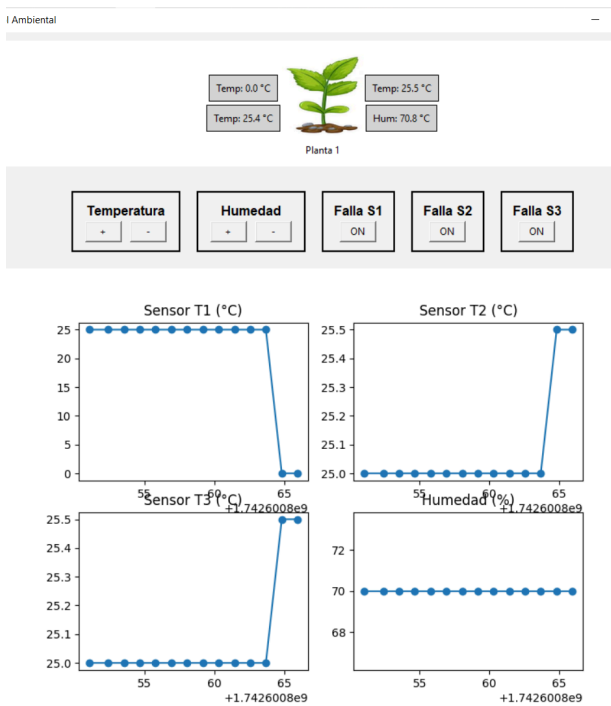



```
executing thought system
-> Sensor system send:
     sensor 1:  25.5
     sensor 2:  25.5
     sensor 3:  25.5
-> Understanding system reads from sensory system

-> Understanding system detect temperature out of range
Desicion system reads from understanding system:   TURNOFF HEATER
['TURNOFF', 'HEATER']
-> Decision system send message:  TURNOFF HEATER
Will system recive  ['TURNOFF', 'HEATER']
-> Will system turn off heater
-> Reason system reads from sensory system
Project1 > intento2.py
```

Case 4: Sensor 1 Failed

As we can see, when conditions are ideal, all sensors show the same temperature. This digital twin includes buttons to simulate a failure in one of the three sensors. The image below shows what happens when we force such a failure. The Understanding System detects temperature out of range and then that there is a high variation in sensors, after that detects that the sensor 1 is the faulty sensor, so the Decision System decides that the sensor 1 needs to be turned off, and finally the Will System Turns Off the Sensor 1.

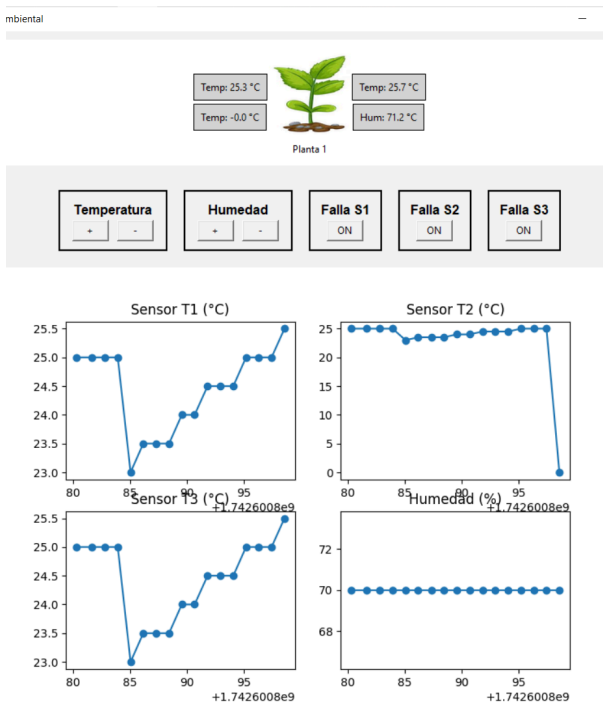



```
-> Understanding system reads from sensory system

-> Understanding system detect temperature out of range
-> Undestanding system detect too much vary in SENSORS
['SENSOR01_OUTLIER', 'SENSOR02_OUTLIER', 'SENSOR03_OUTLIER']
SENSOR01_OUTLIER SENSOR01 OUTLIER
end back propagation
Desicion system reads from understanding system:   OUTLIER SENSOR01
['OUTLIER', 'SENSOR01']
-> Decision system send message:  TURNOFF SENSOR01
Will system recive  ['TURNOFF', 'SENSOR01']
-> Will system turn off sensor 1
-> Reason system reads from sensory system
```

Case 5: Sensor 2 Failed

As we can see, when conditions are ideal, all sensors show the same temperature. This digital twin includes buttons to simulate a failure in one of the three sensors. The image below shows what happens when we force such a failure. The Understanding System detects temperature out of range and then that there is a high variation in sensors, after that detects that the sensor 1 is the faulty sensor, so the Decision System decides that the sensor 2 needs to be turned off, and finally the Will System Turns Off the Sensor 2.

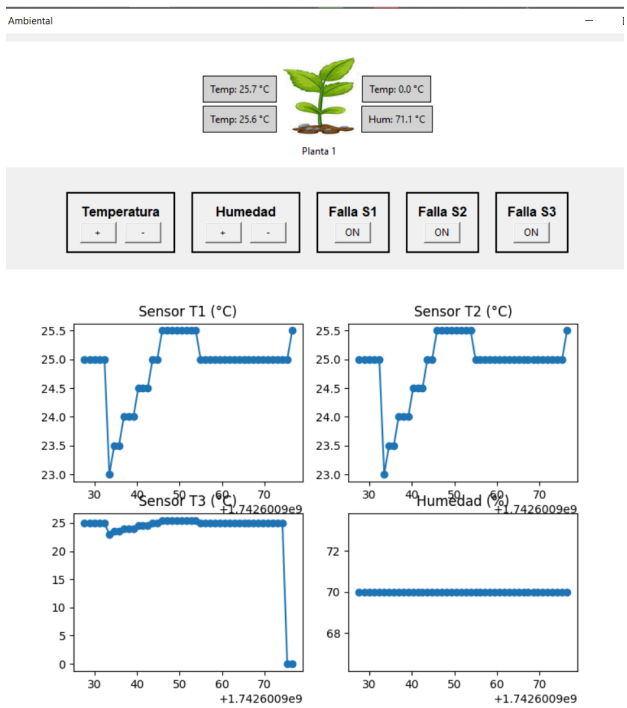



```
-> Understanding system detect temperature out of range
-> Undestanding system detect too much vary in SENSORS
['SENSOR01_OUTLIER', 'SENSOR02_OUTLIER', 'SENSOR03_OUTLIER']
SENSOR01_OUTLIER SENSOR01 OUTLIER
SENSOR02_OUTLIER SENSOR02 OUTLIER
end back propagation
Desicion system reads from understanding system:   OUTLIER SENSOR02
['OUTLIER', 'SENSOR02']
-> Decision system send message:  TURNOFF SENSOR02
Will system recive  ['TURNOFF', 'SENSOR02']
-> Will system turn off sensor 2
-> Reason system reads from sensory system
```

Project1 > intento2.py

Case 6: Sensor 3 Failed (Before)

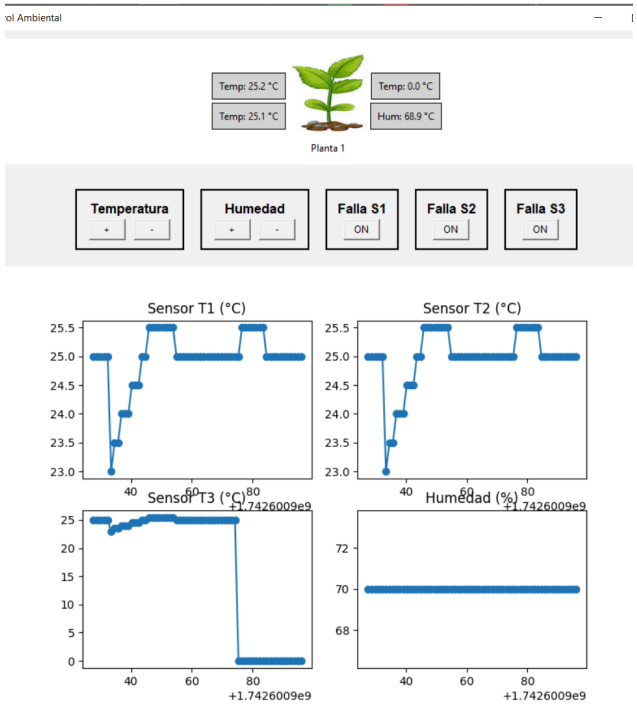
As we can see, when conditions are ideal, all sensors show the same temperature. This digital twin includes buttons to simulate a failure in one of the three sensors. The image below shows what happens when we force such a failure. The Understanding System detects temperature out of range and then that there is a high variation in sensors, after that detects that the sensor 1 is the faulty sensor, so the Decision System decides that the sensor 3 needs to be turned off, and finally the Will System Turns Off the Sensor 3.




```
-> Understanding system detect temperature out of range
-> Undestanding system detect too much vary in SENSORS
['SENSOR01_OUTLIER', 'SENSOR02_OUTLIER', 'SENSOR03_OUTLIER']
SENSOR01_OUTLIER SENSOR01 OUTLIER
SENSOR02_OUTLIER SENSOR02 OUTLIER
SENSOR03_OUTLIER SENSOR03 OUTLIER
end back propagation
Desicion system reads from understanding system:   OUTLIER SENSOR03
['OUTLIER', 'SENSOR03']
-> Decision system send message:  TURNOFF SENSOR03
Will system recive  ['TURNOFF', 'SENSOR03']
-> Will system turn off sensor 3
-> Reason system reads from sensory system
```

Case 6: Sensor 3 Failed (After)

After the sensor is turned off, the sensor 3 reads a 0 value but it is not used for the average temperature.




```
     sensor 3:  -0.0
-> Understanding system reads from sensory system

-> Reason system reads from sensory system

executing thought system
-> Sensor system send:
     sensor 1:  25.0
     sensor 2:  25.0
     sensor 3:  -0.0
-> Understanding system reads from sensory system

-> Reason system reads from sensory system
```

5. Recommendation

- This system works with simulated values, so it could be really nice to obtain the data of real sensors and try to see the behavior of the graphics with this, also it would be great to connect the executions of the system into real life, maybe not to turn on or turn off a heater at the beginning but it could turn on or turn off a LED to simulate this, or light a match to simulate the increase of temperature.
- The project was really hard to implement in this period of time, that is why it was only applied with temperature sensors and a heater, but with more time we can add more type of sensors like luminescence sensors, or control the humidity sensors, also add rules to the Reason System, and more actuators like fans, motors (to add fertilizer or water with a dispenser), humidifier, among other actuators. But with more sensors and actuators our system must be more “intelligent”.